1. **Цель работы**

   Получения навыков по серверному программированию и управлению доступом в базе данных.

2. **Задание**

   1)Создать в БД хранимые процедуры, реализующие:

   — вставку с пополнением справочников(получаем ссылку на внешний ключ по значению данного из родительской таблицы);

   — хранимую функцию любого назначения, соответствующего предметной области .

   — каскадное удаление : удаление всех зависимых данных (перед удалением родителя удаляется цепочка от всех зависимых таблиц);

   2)Создать пользователя или роль с пользователем, который может добавлять данные в базу только через процедуры, но не имеет доступа для редактирования самих таблиц

   Согласно варианту №6:

   Предметная область поликлиника: прием, пациент, процедура/прием, врач, стоимость

   а. кабинеты, в которых проводились любые процедуры, с аппаратурой, в названии которой есть слово «магнит», но на него название не заканчивается

   б. врачи, проводившие прием двух разных пациентов в одном кабинете в один день

   в. аппаратура, не используемая ни в одной процедуре

   г. процедуры, с наименьшей стоимостью среди всех процедур, проводимых в 101 кабинете

   д. кабинеты, в которых проводилось меньше среднего процедур е. пациент, которому проводились процедуры, со всеми аппаратами, начинающимися на букву «Д» (например, дарсонваль, Дерматоскоп)

   ж. врачи, проводившие процедуры с аппаратами магнитотерапии, но не проводившие приемов в 102 кабинете.

3. **Физическая модель**

   Разработанная физическая модель показана на рисунке 1.

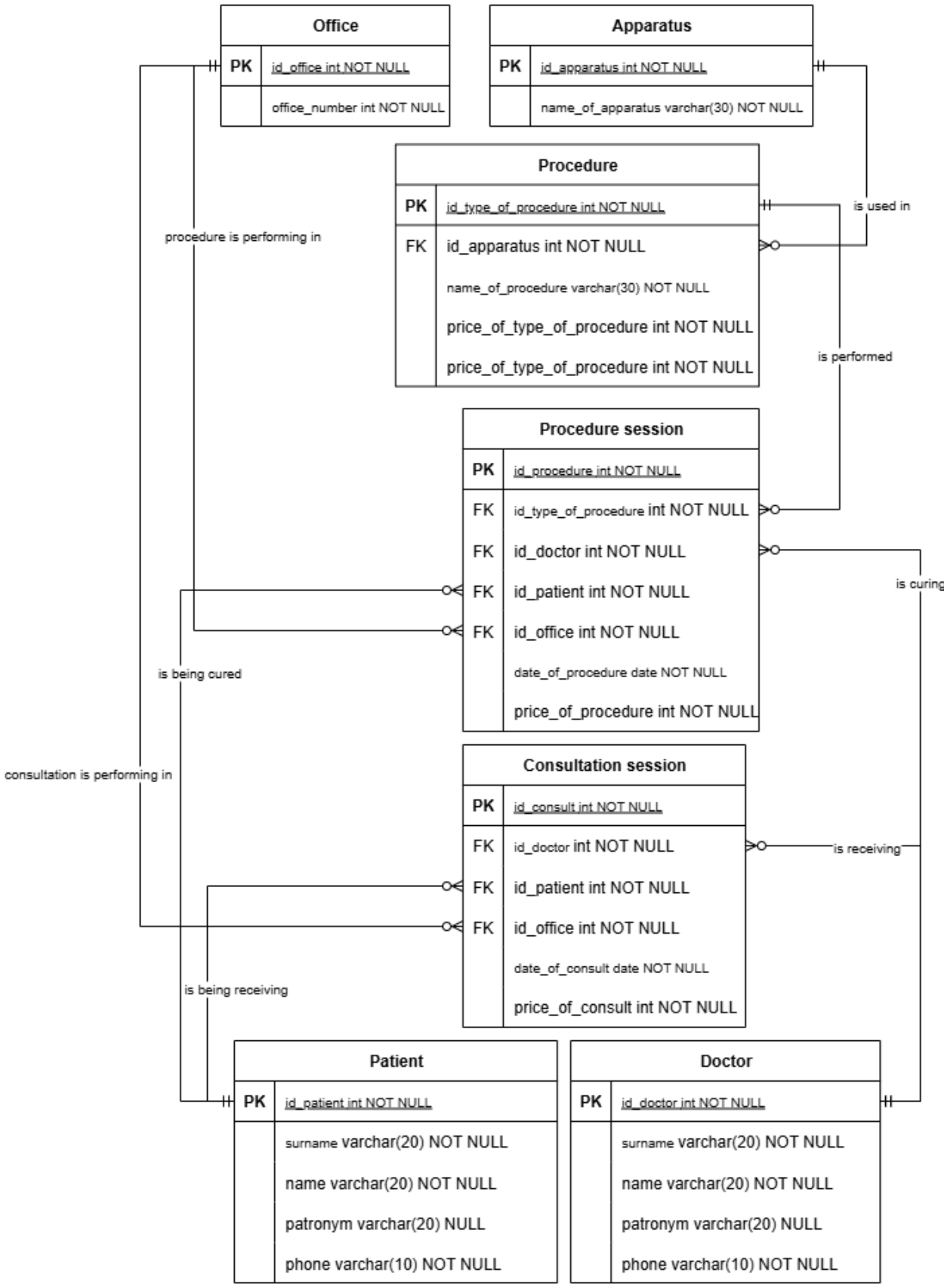


Рисунок 1 – Физическая модель

4. Назначение хранимых процедур и функций

В процессе работы реализованы хранимые процедуры:

- InsertProcedureType – Вставка с пополнением справочника, данная процедура вставляет данные о процедуре и её аппарате, а так же пополняет информацию об аппарате, если его не существует.
- DeletePatient – Каскадное удаление данных, данная процедура удаляет пациента, а так же связанные с ним процедуры и консультации

В процессе работы бала реализована хранимая функция GetConsultPricesForPatient – Подсчёт суммы всех консультаций для данного пациента.

5. **Скрипт для создания хранимых процедур или функций**

```
Delimiter //

create definer=`root`@`localhost` procedure InsertProcedureType(
name_procedure varchar(30),
name_apparatus varchar(30),
price_of_procedure int,
out procedure_id int
)
begin

declare apparatus_id int;
select id_type_of_procedure into procedure_id from TProcedure where
name_of_procedure=name_procedure;

if procedure_id is null then

 select id_apparatus into apparatus_id from Apparatus where
name_of_apparatus=name_apparatus;

 if apparatus_id is null then
        insert into Apparatus (name_of_apparatus) values (name_apparatus);
        set apparatus_id = last_insert_id();
 end if;

 insert into TProcedure(id_apparatus, name_of_procedure, price_of_type_of_procedure)
values (apparatus_id, name_procedure, price_of_procedure);
```

```
 set procedure_id = last_insert_id();

end if;

end //

Delimiter ;

Delimiter //

create definer=`root`@`localhost` function GetConsultPricesForPatient(
 patient_id int
)
returns int
reads sql data
begin

declare consult_prices int;
declare p_name varchar(20);
select name into p_name from Patient where id_patient=patient_id;

if p_name is not null then
   select sum(price_of_consult) into consult_prices from Consultation_Session where
id_patient=patient_id;
   if consult_prices is null then
         set consult_prices = 0;
   end if;
end if;

return consult_prices;

end //

Delimiter ;
```

```
Delimiter //

create definer=`root`@`localhost` procedure DeletePatient(
patient_id int
)
begin

declare p_name varchar(20);
select name into p_name from Patient where id_patient=patient_id;

if p_name is not null then

   delete from Procedure_Session where id_patient = patient_id;

   delete from Consultation_Session where id_patient = patient_id;

   delete from Patient where id_patient = patient_id;

end if;

end //

Delimiter ;
```

6. **Скрипт создания пользователя**

```
drop user if exists 'new_user'@'localhost';

create user 'new_user'@'localhost' identified by '123';
grant select on hospital_info.* to 'new_user'@'localhost';
grant execute on procedure hospital_info.InsertProcedureType to 'new_user'@'localhost';
grant execute on function hospital_info.GetConsultPricesForPatient to
'new_user'@'localhost';
grant execute on procedure hospital_info.DeletePatient to 'new_user'@'localhost';
```

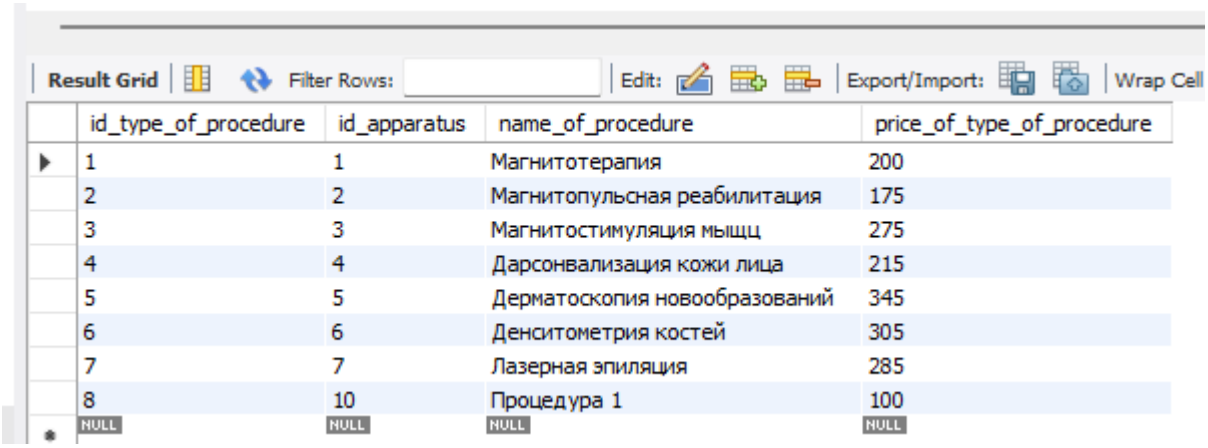
7. **Иллюстрации работы процедур**

```
use hospital_info;
```

call InsertProcedureType("Процедура 1", "Аппарат 1", 100, @proc_id);

select @proc_id;

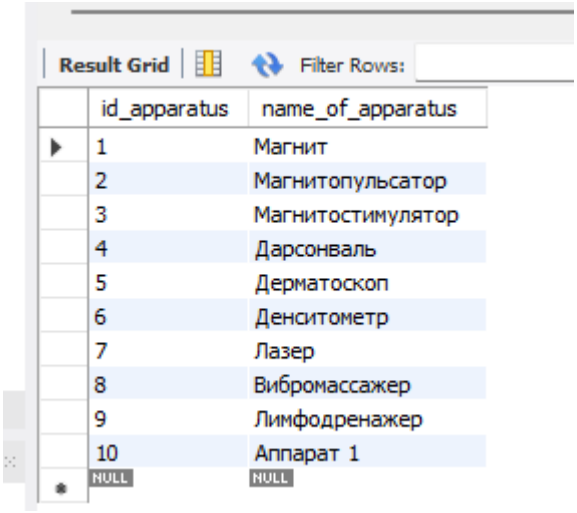
Таблицы данных, участвующие в процедуре, показаны на рисунке 2-3.



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

id_type_of_procedure	id_apparatus	name_of_procedure	price_of_type_of_procedure
1	1	Магнитотерапия	200
2	2	Магнитопульсная реабилитация	175
3	3	Магнитостимуляция мыщц	275
4	4	Дарсонвализация кожи лица	215
5	5	Дерматоскопия новообразований	345
6	6	Денситометрия костей	305
7	7	Лазерная эпиляция	285
8	10	Процедура 1	100
NULL	NULL	NULL	NULL

Рисунок 2 – Таблица типа процедур



Result Grid | Filter Rows:

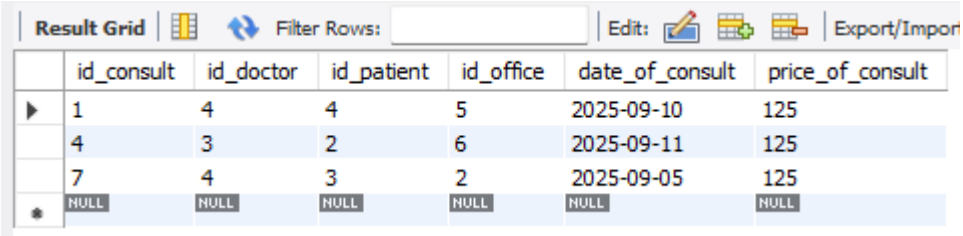
id_apparatus	name_of_apparatus
1	Магнит
2	Магнитопульсатор
3	Магнитостимулятор
4	Дарсонваль
5	Дерматоскоп
6	Денситометр
7	Лазер
8	Вибромассажер
9	Лимфодренажер
10	Аппарат 1
NULL	NULL

Рисунок 3 – Таблица аппаратов

use hospital_info;

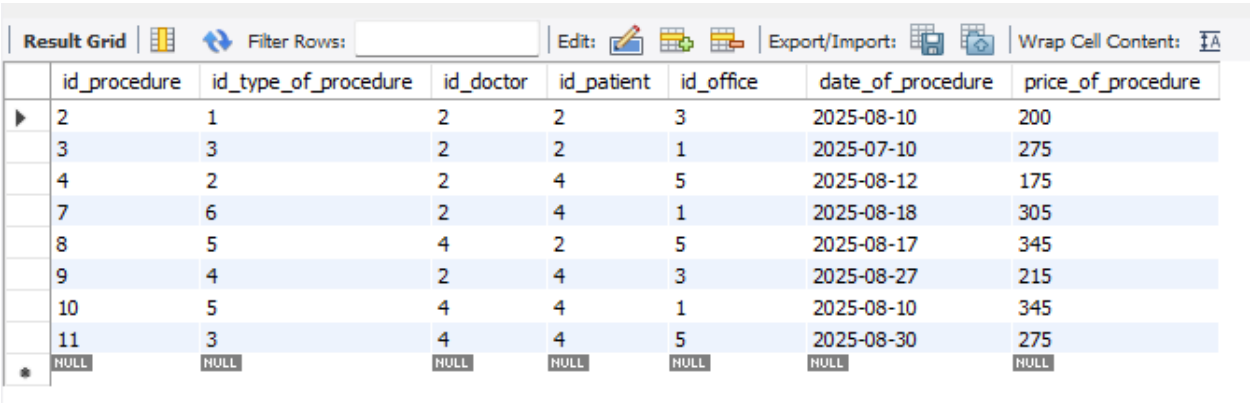
call DeletePatient(1);

Таблицы данных, участвующие в процедуре, после выполнения процедуры показаны на рисунках 3-5.



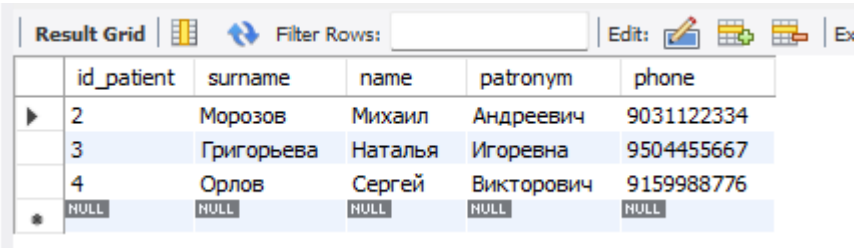
id_consult	id_doctor	id_patient	id_office	date_of_consult	price_of_consult
1	4	4	5	2025-09-10	125
4	3	2	6	2025-09-11	125
7	4	3	2	2025-09-05	125
NULL	NULL	NULL	NULL	NULL	NULL

Рисунок 3 – Таблица консультаций



id_procedure	id_type_of_procedure	id_doctor	id_patient	id_office	date_of_procedure	price_of_procedure
2	1	2	2	3	2025-08-10	200
3	3	2	2	1	2025-07-10	275
4	2	2	4	5	2025-08-12	175
7	6	2	4	1	2025-08-18	305
8	5	4	2	5	2025-08-17	345
9	4	2	4	3	2025-08-27	215
10	5	4	4	1	2025-08-10	345
11	3	4	4	5	2025-08-30	275
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Рисунок 4 – Таблица процедур



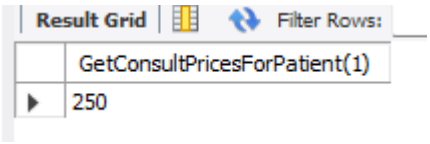
id_patient	surname	name	patronym	phone
2	Морозов	Михаил	Андреевич	9031122334
3	Григорьева	Наталья	Игоревна	9504455667
4	Орлов	Сергей	Викторович	9159988776
NULL	NULL	NULL	NULL	NULL

Рисунок 5 – Таблица пациентов

use hospital_info;

select GetConsultPricesForPatient(1);

Результат процедуры показан на рисунке 6



GetConsultPricesForPatient(1)
250

Рисунок 6 – Результат работы процедуры

Таблица, участвующая в процедуре, изображена на рисунке 7.

id_consult	id_doctor	id_patient	id_office	date_of_consult	price_of_consult
1	4	4	5	2025-09-10	125
2	4	1	5	2025-09-10	125
3	3	5	6	2025-09-11	125
4	3	2	6	2025-09-11	125
5	3	1	6	2025-09-11	125
6	1	5	2	2025-09-09	125
7	4	3	2	2025-09-05	125
NULL	NULL	NULL	NULL	NULL	NULL

Рисунок 7 – Таблица консультаций

8. Иллюстрация работы прав доступа пользователя

Скриншот, отображающий права пользователя, показаны на рисунке 8.

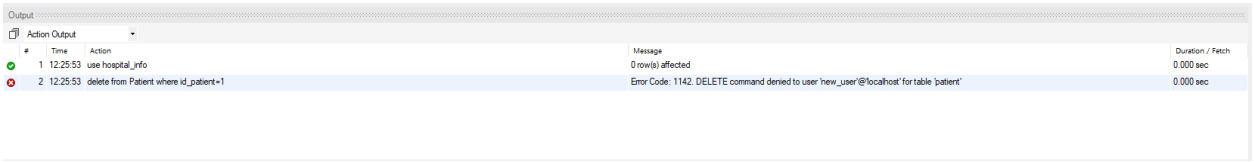


Рисунок 8 – Демонстрация прав пользователя

9. Выводы

В процессе лабораторной работы были получены практические навыки создания хранимых процедур и функций в СУБД MySQL, которые реализуются в общем смысле похоже, но различаются количеством возвращаемых значений. Были получены навыки управления доступом и создания новых пользователей с правами на совершение определённых действий.